

MINOR PROJECT 2

Findings

Security Analysis, Test Results & Recommendations

Project	Zero Trust Architecture — Web Application
Student	Himanshu Soni
Organisation	Naviotech Solutions Pvt. Ltd.
Division	Cybersecurity
Standard	NIST SP 800-207
Year	2025

1. Executive Summary

The Zero Trust Architecture web application successfully demonstrates the core ZTA principles of continuous verification and least-privilege access in a containerised web environment. All seven planned security test scenarios passed. The implementation effectively prevents privilege escalation, unauthenticated bypass, and user enumeration attacks. Two partial gaps — hardcoded credentials and absent MFA — have been identified and are addressed in the recommendations section.

2. Security Test Results

Test	Attack Vector	Result	ZTA Principle
Admin login	Valid auth flow	PASS	Always Verify
Employee login	Valid auth flow	PASS	Always Verify
Privilege escalation	Employee → admin.html	BLOCKED	Least Privilege
Direct URL bypass	Unauth → admin.html	BLOCKED	Always Verify
Invalid credentials	Wrong password	BLOCKED	Minimal Attack Surface
Brute-force simulation	Repeated wrong attempts	BLOCKED	Minimal Attack Surface
Session validation	Role check per page load	PASS	Continuous Monitoring

3. ZTA Compliance Findings

3.1 Strengths

- Single Entry Point enforced: No role can be selected by the client — it is assigned exclusively server-side after credential validation.
- Micro-segmentation effective: Admin and Employee dashboards share no components. An Employee holding a valid session cannot access Admin routes.
- Generic error messaging prevents user enumeration: All invalid login attempts return the same response regardless of whether the username exists.
- Docker containerisation provides OS-level isolation: The web server process has no direct access to host resources.
- Least-privilege enforced at dashboard level: Each role sees only its own set of permitted actions.

3.2 Gaps Identified

Gap	Risk	Description	Recommendation
Client-side credential store	High	Credentials hardcoded in login.html JS — visible in browser dev tools	Move to server-side auth with bcrypt-hashed DB credentials

localStorage manipulation	role	Medium	A user can manually set localStorage role to 'admin' in dev tools	Replace with server-issued signed JWT (HttpOnly, Secure cookie)
No MFA		Medium	Single factor (password) only — no TOTP or push notification	Implement TOTP-based MFA using RFC 6238
No HTTPS/TLS		High	Traffic unencrypted in demo deployment	Add TLS via Let's Encrypt or self-signed cert in Nginx
No persistent audit log		Low	Access events shown via alert() only, not persisted to DB	Log to MySQL table: timestamp, username, IP, outcome

4. Access Control Effectiveness

The following matrix summarises access control outcomes across all role-resource combinations tested during the evaluation phase:

Resource	Admin (authenticated)	Employee (authenticated)	Unauthenticated
login.html	Accessible	Accessible	Accessible
admin.html	Granted	DENIED — redirect to login	DENIED — redirect to login
employee.html	DENIED — redirect to login	Granted	DENIED — redirect to login
Direct URL (any)	Allowed if role matches	Allowed if role matches	All denied — redirect

5. Vulnerability Analysis

5.1 OWASP Top 10 Coverage

OWASP Risk	Status	Notes
A01 — Broken Access Control	Mitigated	Role enforced on every protected page load
A02 — Cryptographic Failures	Partial	No HTTPS in demo; planned in roadmap
A03 — Injection	N/A (demo)	No server-side DB queries in current scope
A04 — Insecure Design	Partial	Client-side auth is a design gap; noted for production
A05 — Security Misconfiguration	Mitigated	Nginx default config, no directory listing
A07 — Identification & Auth Failures	Partial	No MFA, no account lockout yet

A09 — Security Logging & Monitoring	Partial	Alert-based only; DB logging in roadmap
-------------------------------------	---------	---

6. Recommendations

6.1 Immediate (Production Hardening)

- Migrate authentication to a server-side API endpoint that validates credentials against a MySQL table with bcrypt-hashed passwords.
- Replace localStorage session with an HttpOnly, Secure, SameSite=Strict cookie holding a signed JWT (RS256) with a 15-minute expiry.
- Enable HTTPS via Nginx TLS configuration with a Let's Encrypt certificate to encrypt all traffic in transit.

6.2 Short-Term Enhancements

- Implement TOTP-based MFA (RFC 6238) as a second authentication factor for all roles.
- Add login attempt logging to the database (table: audit_log with columns: id, username, ip, timestamp, outcome).
- Implement account lockout after 5 consecutive failed attempts with exponential backoff.

6.3 Long-Term / Enterprise Scale

- Integrate a centralised Identity Provider (IdP) using OAuth 2.0 / OpenID Connect (e.g. Keycloak or Auth0) for SSO.
- Deploy a SIEM dashboard (e.g. Elasticsearch + Kibana) to aggregate and visualise all access events in real time.
- Implement device posture checking (OS patch level, certificate validation) via an MDM integration.
- Introduce micro-segmentation at the network layer using a service mesh (e.g. Istio) for mTLS between internal services.

7. Conclusion

The Zero Trust Architecture proof-of-concept successfully validates the core ZTA principles in a containerised web application. All role-based access control requirements were met, and all seven security test scenarios passed. The identified gaps are inherent to the demo scope and are fully documented in the roadmap. The project provides a solid foundation for a production-grade ZTA system and demonstrates a clear understanding of the NIST SP 800-207 framework, making it a strong academic and portfolio deliverable.